

Parallelism as a Consequence of Normalized Behavior

The Frozen Frame: One Verb, One Window, One Writer

Registry: [\[HOWL-COMP-16-2026\]](#)

Series Path: [\[HOWL-COMP-1-2026\]](#) → ... → [\[HOWL-COMP-12-2026\]](#) → [\[HOWL-INFO-11-2026\]](#) → ... → [\[HOWL-INFO-14-2026\]](#) → [\[HOWL-INFO-15-2026\]](#) → [\[HOWL-INFO-16-2026\]](#) → [\[HOWL-INFO-17-2026\]](#) → [\[HOWL-COMP-16-2026\]](#)

DOI: 10.5281/zenodo.22585474

Date: September 2026

Domain: Computer Architecture / Concurrent Execution

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Claude Fable 5.1.

Abstract

A prior paper in this series, *The General Theory of State Change* [\[HOWL-INFO-17-2026\]](#), normalized behavior: it reduced all state change to one verb — a guarded, staged, recorded movement of quantities between addresses — applied by a single closed interpreter with no path around it. That paper argued its rules from correctness: whole-set atomicity, a complete audit ledger, validation that cannot be bypassed.

This paper states a consequence the prior paper left implicit. Two of its rules — staged delivery and nonsubversion — jointly determine not only *what* a change is but *when memory changes*. Any realization honoring both rules acquires a two-phase execution structure: a serialized mutation window in which the single executor applies all staged change, followed by a read phase in which the entire world is immutable. This structure, which we name the frozen frame, is a complete memory model, a parallelism strategy, and an elimination of concurrency overhead — obtained as a corollary rather than built as a feature. Four of the five components of the concurrency tax [\[HOWL-INFO-14-2026\]](#) go to zero structurally; the fifth becomes an explicit, tunable data-layout variable, which is exactly the variable data-oriented design practice knows how to tune. The paper derives the corollary, accounts for the tax, describes the execution strategies the freeze permits, draws the boundary of what does not become data, states the costs honestly, and closes with falsifiable claims. The claims are of three kinds, tagged throughout: the phase structure and its properties are theory, holding for any realization of the rules; the batching, placement, and thread substrate described are engineering, illustrative of one running realization; the closing claims are the test.

1. The problem: mutation is distributed

(Theory.)

Every practitioner of concurrent software pays the same costs. Locks that serialize what was supposed to run in parallel. Atomic operations and memory-ordering annotations whose correct use is a specialist skill. Job systems with dependency graphs that must be declared, maintained, and believed. Data races that survive every test run until the one that matters. Deadlocks, priority inversions, torn reads, and the permanent low-grade fear that two threads are touching the same bytes.

These appear to be many problems. They are one problem. Conventional software distributes its writes across every function that mutates state. An application is a large set of verbs — hand-written functions, each of which validates, mutates, and records on its own authority, at whatever moment it happens to run. When the mutation points are many and unscheduled, any read may race any write, and the entire apparatus of concurrency control exists to manage that interleaving. Locks manage it by exclusion. Atomics manage it by hardware arbitration. Memory models manage it by defining which interleavings are observable. Job graphs manage it by ordering the verbs. All of it is management. None of it is removal.

This paper asks the other question: what happens to concurrency if the mutation points are reduced to one, and that one is scheduled? The answer is that the interleaving does not need to be managed, because it no longer exists — and the machinery built to manage it becomes unnecessary in the strict sense, not merely avoidable. The reduction to one scheduled mutation point is not proposed here as a design technique. It is inherited, already argued and already running, from a prior result in this series. The present paper's contribution is to show that the inherited result casts a shadow in physical time, and that the shadow is a parallel execution model of unusual simplicity.

2. The inherited result: behavior normalized

(Theory, imported. This section makes the paper self-contained; the full argument is in [HOWL-INFO-17-2026].)

The prior paper's claim is that behavior — state change — was never normalized the way data was. Codd normalized data at rest: every fact became a value in a table, stored once, addressed by key, and a defect class died. The symmetric event never happened for state change, so applications remained collections of bespoke verbs. The prior paper supplies the missing normalization. This paper needs five of its results, introduced here in the minimum form the derivation requires.

One verb. All state is quantities at addresses: a number, of a named kind, at a reachable location. All change is movement of quantities between addresses. A change is carried as a record — called a conversion set — which contains its own validity conditions: costs that must be payable and requirements that must hold. Validity is a property of the change record, never of the code path that requested it.

Staging. Deciding a change and applying it are different moments. The point of decision

writes an intent record — called an envelope — naming who, to whom, which conversion set, and when. A later delivery window validates and applies staged envelopes as whole sets, atomically: applied entirely or refused entirely, never partially. This is the prior paper’s Rule 7.

The single closed interpreter. One validator decides all validity. One executor applies all change. There is no path around them — not for authored content, and not for the system’s own internal phases, which consume the same functions. A second executor, or a code path that writes state directly, disqualifies the realization. This is the prior paper’s Rule 12, its nonsubversion rule, and it is the load-bearing discipline of everything that follows.

The ledger. Because every change passes through one executor, recording is a property of the executor, and nothing can change state without leaving a record. The system’s history is an append-only transaction log of every attempted movement — including refused movements, with the reason for refusal — each entry carrying its phase, a monotonic order, and its provenance.

Flat, self-describing records. State is records of fixed shapes whose leaves are numbers and ids. No meaning is welded into the interpreter; domain concepts exist only as name records attached to addresses. A record set is complete and self-describing: copying it is saving it, and it does not know or care where it runs. Absence is a uniform, inert sentinel value, so a partially authored or partially copied record set is valid everywhere.

The prior paper argued these rules from correctness. Duplicated validation, bypassed checks, unaudited writes, and decision layers that disagree with the rules they act under all become structurally impossible, because the redundant copies that could disagree no longer exist. That argument is complete on its own terms, and this paper does not repeat it. This paper observes something the correctness argument never needed to say aloud: the same two rules that guarantee atomicity and audit also determine, exactly, *when memory is allowed to change*. That determination is worth a paper, because it is a concurrency result, and it was purchased without anyone buying it.

3. The corollary: the two-phase frame

(Theory. The paper’s central claim.)

The claim, stated first and derived after:

Any realization honoring staged delivery (Rule 7) and nonsubversion (Rule 12) acquires a two-phase execution structure at the period its delivery windows define: a serialized mutation window, in which the single executor applies all staged envelopes and completes the ledger, followed by a read phase, in which all shared state is immutable.

The derivation is four steps, each verifiable independently.

First: only the executor writes. This is Rule 12 directly. No code may move a value except through the executor — not gameplay code, not the user interface, not the engine’s own phases. If the rule holds, the set of writers has exactly one member.

Second: the executor runs only in the delivery window. This is Rule 7 directly. Change is staged at the point of decision and applied in a known window; the executor is the delivery mechanism, and delivery has a schedule.

Third: therefore, outside the window, there is no writer. The one writer that exists is not running.

Fourth: therefore, outside the window, all shared state is frozen. Data that no one can write is immutable for the duration, not by convention, not by discipline, but by the absence of any agent capable of changing it.

One observation completes the structure and closes the loop. Work performed during the read phase does produce results — decisions, computed placements, chosen actions. But under the normalization, those results are not mutations. They are new envelopes: staged intents, appended for the *next* window. Even the outputs of parallel work are data in flight toward a future mutation window, never writes against the present. The frame is therefore closed end to end: the window applies last frame’s intents and seals the ledger; the freeze point is the last write; the read phase computes on frozen state and emits intents; a gather point collects the emissions; the next window begins. Nothing escapes the cycle, because escaping it is precisely what Rule 12 forbids.

We name this structure the **frozen frame**.

Two properties of the frame deserve immediate statement, because readers will otherwise supply narrower assumptions than the claim makes.

The period is a parameter, not architecture. In an interactive game, the window runs at the top of each rendered frame — sixteen milliseconds at sixty frames per second. In headless simulation, the period is zero in the sense that the next window begins the moment the current read phase completes; the structure runs as fast as frames can be computed. In an enterprise realization, the window could run nightly, as batch posting windows already do. The phase structure is invariant under the choice of clock. The prior paper’s realization already demonstrates period-as-parameter in miniature: one constant converts every duration in the system between turn-based and real-time interpretation, and the frame period is the same kind of value.

The freeze applies to shared state. Workers require private scratch memory, and private memory is outside the claim, because immutability is a property needed exactly where more than one agent can look. The precise statement — immutability of *shared* state — matters, and Section 8 returns to it as a scope condition on real realizations.

4. The frozen frame is a memory model

(*Theory.*)

A memory model is the contract that makes concurrent programs meaningful. It must answer three questions. Visibility: when does one agent’s write become observable to another? Ordering: in what sequence do writes become observable? Atomicity of observation: can an agent see a value halfway through being written — a torn read? Conventional systems answer these questions with hardware fences, cache-coherence protocols,

and language-level atomic types, and the answers are paid for per access, in both machine cost and program complexity. The specialist difficulty of concurrent programming is almost entirely the difficulty of using these answers correctly.

The frozen frame answers all three questions by construction, for the entire read phase. When there is no writer, visibility is total: everything observable is the complete, final output of the last window, everywhere, for every agent. Ordering is irrelevant: there are no writes to order. Tearing is impossible: no value is ever halfway through anything. These are not guarantees enforced by machinery; they are vacuous truths, in the way that a room with no doors needs no locks. During the read phase, any core, any thread, any node may read anything, in any order, at any rate, with no coordination protocol whatsoever, and the program is correct.

What remains of synchronization is countable on one hand. The process requires two barriers per frame: wait-for-freeze, which holds workers until the mutation window has closed, and wait-for-gather, which holds the next window until workers have emitted their envelopes. It additionally requires one shutdown flag, polled cooperatively. That is the complete synchronization inventory, and its cost is constant per frame — it does not grow with core count, with worker count, or with the number of resources in the world. Conventional coordination cost scales with the contention structure of the workload; here the contention structure has been deleted, and the residual coordination is two fence posts around a schedule.

The governing image for this structure is not the conventional software stack at all. It is the digital signal processor. A DSP receives a buffer of samples, applies one deterministic transform pass, and outputs a buffer that is inert until the next pass; between passes, the data simply sits, complete and readable. The frozen frame is that model applied to a world. The envelope pass is the transform. The frozen state plus the sealed ledger is the output buffer. The frame period is the sample clock. Software becomes signal processing at exactly the moment its mutation collapses into a scheduled pass — and the normalization is what performs the collapse, by leaving the system only one verb to schedule.

One further property arrives free and deserves its own paragraph, because it replaces an entire pattern family. The transaction ledger is completed inside the window and frozen with everything else. Every attempted change of the frame — applied or refused, with phase, order, and reason — is therefore readable, lock-free, by any worker during the read phase. The question “what changed this frame, and why” is a query on immutable data. Systems that need reactivity read the ledger instead of subscribing to events. The observer pattern, with its registration lifetimes, its callback re-entrancy hazards, and its hidden coupling between publisher and subscriber, is replaced by reading a log that cannot change while being read. Recording was a structural consequence of the single executor; notification turns out to be a structural consequence of recording plus the freeze.

5. The concurrency-tax accounting

(Theory; imports one unit of measure.)

The series’ information-processing theory [[HOWL-INFO-14-2026](#)] provides the instru-

ment for measuring what the frozen frame is worth. That paper defines the **concurrency tax**: the overhead a workload pays for executing beside other workloads rather than in isolation, decomposed into five components. *Contention* — the resource you need is held by someone else, and your operation’s duration inflates by the wait. *Blocking* — your pipeline is fully idle on a critical resource. *Coordination* — operations spent managing shared access itself: lock protocols, handoffs, merge resolution. *Interleave* — operations spent deciding which stream to service next. *Cascade* — work destroyed by another stream’s activity: evicted caches, invalidated locality, the re-orientation cost after interruption. The same paper shows the tax is derivable from the contention graph — the topology of shared resources and the streams competing for them — and that its scaling law follows the topology.

Run the five components against the frozen frame, phase by phase.

Contention in the read phase is zero by construction. Contention is waiting for a held resource, and holding is a concept that applies only to mutable resources; an immutable world has nothing to hold and nothing to wait for. In the mutation window, contention is absent by definition rather than by construction: the window is one serialized stream, and a single stream cannot contend with itself.

Blocking follows the same accounting. In the read phase there is no critical resource to be idle on. In the window, the single stream is the schedule.

Coordination collapses to the two barriers of Section 4, constant per frame. No per-resource negotiation exists because no resource needs negotiating; no handoff protocol exists because nothing is handed off — workers read a world and append to a queue.

Interleave — the continuous triage of “what do I service next” that consumes schedulers and humans alike — becomes one batch-dispatch decision per frame, made once, at the freeze point, when work is assigned to workers. Inside the window, the ledger’s monotonic order *is* the schedule, decided by arrival rather than deliberation.

Cascade survives. When a working set moves between cores or memory nodes, locality is destroyed and rebuilt, and that cost is real: cache lines are refetched, remote memory is slower than local. This is the one tax component the architecture does not eliminate.

But look at what the architecture has done to it. In a conventional system, cascade is *emergent* — a context switch evicts your cache at a moment chosen by a scheduler you don’t control, interleaved with mutation you can’t predict. In the frozen frame, cascade is an explicit variable: working sets move only when the dispatch decision moves them, over data that cannot change mid-flight, at a frame boundary you chose. The component has been converted from an ambient cost into a layout and placement problem — and layout and placement problems are precisely what an existing engineering practice knows how to solve, as Section 6 takes up.

In the contention-graph vocabulary: the read phase has no contention graph at all, because it has no shared mutable resources; and the frame as a whole is a star topology with exactly one scheduled visit to the center per period. The star with continuous competition is the worst topology, with divergent tax growth. The star with one visit per period is the

degenerate case whose tax does not grow with stream count. The frozen frame does not find a better position on the tax curve; it exits the curve.

This closes the loop opened in Section 1, and the closure is worth stating in the series' own terms. The prior paper diagnosed conventional software's defect as verb redundancy: each kind of change defined many times, privately, in mutually ignorant callers. This paper's accounting shows that the concurrency tax is verb redundancy *measured in a different unit*. Many verbs means many mutation points; many mutation points means interleaved reads and writes; interleaving is the sole reason any tax component except cascade exists. One verb, one window, one writer — and four of five components are not reduced but gone.

6. What the freeze permits: execution strategies

(Engineering. The register shifts here. Everything before this section follows from the rules and holds for any realization. This section describes one realization — a Zig game engine running the normalized-behavior interpreter — and is illustrative. The paper teaches the operation; the code merely evidences that the operation runs.)

6.1 Batches on frozen data

The unit of parallel work is the batch: a set of records read from the frozen world, processed by one worker, emitting envelopes. A batch might be the considerations of five hundred actors to score, a heat-map field to smooth, a region of tiles to evaluate, a slice of the ledger to analyze.

The starting layout is an array of structs per batch — the records as they are, contiguous, walked linearly. From there, layout is tuned per workload: hot fields split from cold, arrays of structs converted to structures of arrays where a pass touches one field of many records, batches sorted to match access order. This is data-oriented design — the practice, associated with Mike Acton and the performance culture around it, of organizing memory for the transform that will read it rather than for the object model that describes it — and one sentence locates this paper's relationship to it. Data-oriented design tells you to organize the data for the transform; the normalization *guarantees the data is organizable*. In a conventional engine, objects resist relayout because their methods, their inheritance, and their aliased references are load-bearing; memory cannot be reshaped without breaking code identity. Here nothing is load-bearing except the bytes and the schema. Records are numbers and ids of fixed shapes; meaning lives in the ids and resolves the same from any address. Layout is a free variable, and the freeze makes every relayout safe, because the source of a copy cannot change under the copier. The frozen frame is the bridge between the prior paper's theory and the data-oriented practitioner's craft: the theory produces, as a side effect, exactly the memory regime the craft has always wanted and has had to fight object-oriented architecture to approximate.

6.2 Placement: cores and NUMA nodes

On multi-socket and chiplet machines, memory is not uniformly distant: each core reaches its local memory node faster than remote nodes. Non-uniform memory access

— NUMA — makes the placement of work near its data the dominant remaining performance variable, which is to say: placement is where the surviving cascade component of Section 5 is paid down.

The frozen frame admits two placement strategies, and the realization uses both.

The first is dedicated infrastructure per core. Long-lived workers each own a core or set of cores and a standing responsibility: one runs the heat-map smoothing passes, one runs path searches, one runs ledger analytics, one prepares render data. Each reads frozen state, computes, and emits envelopes. Because the read phase requires no coordination, these workers share nothing but the two barriers; they are, for the duration of every read phase, programs running against a read-only database that happens to be the world.

The second is copy-out, copy-back. A batch’s working set is copied to a remote node’s local memory, processed there at local speed, and its results return as envelopes. The safety of this maneuver is not an engineering achievement; it is the prior paper’s distribution-independence property (Rule 11) applied at small scale. A record set is complete and self-describing — copying a scene *is* saving it — and a record’s meaning is carried by group ids and record ids that resolve identically anywhere. A batch of values or a heat map’s chunks is bytes whose meaning travels with them. No pointer into a live object crosses the boundary, because no such pointer exists; no callback crosses, because there are none; the inert-absence sentinel travels as bytes like everything else, so even a partial working set is valid on arrival. NUMA transfer is the small case of the same property that lets scenes replicate across processes and machines.

6.3 The thread substrate

What the model demands from the operating-system layer is short enough to enumerate in one sentence: spawn a worker, pin it to a node and core set, let it poll a shutdown flag, and drain it at a batch boundary. The realization’s thread module is a few hundred lines: creation, NUMA-node and core-affinity pinning through the platform’s processor-group interfaces, a polled cooperative drain with a timeout, and a forced-termination fallback. It contains no locks, no condition variables, no channels, and no synchronization vocabulary at all, because the frame structure gives it nothing to synchronize.

The thinness of this interface is itself evidence. A thread layer that needed rich configuration — priorities negotiated per resource, wake conditions, work-stealing protocols — would indicate work semantics leaking into placement. This one carries topology and nothing else, which is what a correctly drawn boundary looks like from the substrate side.

One corollary of staging deserves statement here because it lands in the thread layer: cancellation is safe for the same reason reading is. A worker’s outputs are staged envelopes, not mutations, so a worker that checks the shutdown flag between batches and exits abandons nothing mid-mutation — there is no mid-mutation. Cooperative shutdown is therefore *sufficient* under the model, completing within one batch duration, and forced termination survives in the substrate only as defense against a hung batch — a stuck path search, a substrate bug — never as a correctness mechanism.

6.4 Determinism and replay

(Scoped, with its condition stated.)

The mutation window is serialized, and the ledger’s order is total. A frame’s output is therefore a function of three inputs: the prior frozen state, the set of envelopes delivered, and the state of the random stream. When all three are captured — and the third is the operative condition, since the record shapes include authored randomness — execution replays identically from a snapshot, and divergence between two runs is itself a detectable defect signal. The structure *permits* replay-identical execution; the present realization treats randomness-as-state as engineering in progress and therefore claims the property conditionally. The honest formulation: determinism is a consequence of the frozen frame given RNG-as-state, and a realization that ships the given gets the consequence.

7. The boundary: what does not become data

(Theory and engineering, marked per subsection. This section prevents the strong misreading — that the frozen frame is pressure to normalize everything. It is the opposite: the frame works because the substrate stayed code.)

7.1 The substrate signature *(engineering, generalizing to a pattern)*

Every subsystem this realization excludes from the record layer exhibits one two-part signature. On the record side: direction — ids naming what to read, scales and curves shaping how to weigh it, filters classifying what counts. On the code side: execution — the loop, the search, the storage walk.

The realization’s spatial system is the worked example. A heat map — a per-tile scalar field used for placement and movement decisions — is defined almost entirely as rows. Its smoothing behavior is fields: whether falloff blends from all adjacent tiles or the highest, the falloff percentage, the radius. Its inputs are import records, each naming a tile layer and declaring, as data, how that layer’s contents enter the field: priority-wins or additive, actor footprints written as indices or as constants, a per-import scale whose sign alone converts attraction to repulsion, and a filter that admits only tiles carrying a named conversion set — so the same machinery yields “the wheat field” or “the defense field” by classification through the economy’s own vocabulary. What remains in code is the traversal: visit each tile, apply the authored kernel, walk the chunked storage. The storage itself is a memory layout with a dirty bit — chunk arrays, row-major indexing, change timestamps for cache invalidation — and holds no meaning at all.

The general statement: substrate is decomposed until only *iteration* remains in code. The semantics of every write, every weighting, every filter are extracted into records before the handoff, and the code contributes visit-each-element. Under the frozen frame this decomposition acquires its execution meaning: those iteration units — record-parameterized loops over frozen records — are precisely what Section 6 schedules onto cores. The boundary between rows and code is simultaneously the boundary between what is authored and what is dispatched.

7.2 The vocabulary test (*theory*)

The boundary needs a decision procedure, because a maintainer faces it one value set at a time: this enum in front of me — does it become a resource group, or does it stay code? The test is two questions. **Does the set change with domain, and does it scale?**

Both yes: the set is vocabulary, and it belongs in rows. Render layers pass the test — every game names its own layers, the count is open, and a colony sim’s *GroundPickup* is not a ledger’s anything. Job types, need types, statuses, and material kinds all pass; they are what a domain *is*.

Either no: the set is infrastructure selection, and it stays an enum that code maps. Text justification fails both halves — left, right, center, inherit are the same four values in a colony sim, an accounting ledger, and a pod scheduler, and the fifth value is never coming. The set is closed by the nature of the operation, not by the author’s current needs.

The test carries one clause that keeps it honest: it is relational, not taxonomic. Justification *could* become a group — in a game where alignment is a purchasable property of the world, it is economy vocabulary, and the classification flips. The test evaluates the set *relative to the domains being built*, exactly as the series’ processing entropy is a property of an element relative to a processor rather than of the element alone. The same set can be infrastructure in one engine and vocabulary in another, and both classifications are correct.

7.3 The anti-claim (*theory*)

State plainly what does not migrate to rows, ever, under this model: placement, iteration, storage layout, platform lifecycle — the machine. Threading is infrastructure. NUMA topology is infrastructure. Chunk formats and cache lines are infrastructure. Normalizing them would be rows whose vocabulary never varies with domain and never scales with authoring, paying data’s interpretation cost for none of data’s benefit — normalization theater.

The point is stronger than permission to stop; it is a dependency. The frozen frame is a consequence of normalizing *behavior*, and it schedules record-parameterized loops over frozen records — but the loops, the scheduler, and the machine remain code, permanently, and the frame works *because* they do. The theory of the prior paper delegated substrate explicitly and held only the interface; this paper’s contribution runs through that same delegation: the record layer supplies frozen, relocatable, self-describing work, and the code layer supplies fast, placed, tuned execution. Erase the boundary in either direction and both halves lose what makes them good.

8. Costs and limits

(Theory and engineering, honest. The section a skeptical reader turns to first, written for that reader.)

The serialized window bounds throughput. The mutation window is one stream, and its duration grows with the number of envelopes delivered and the size of their conversion

sets. Under heavy staged load, the window is the frame-rate ceiling. The mitigation is available but conditional: the window can be internally parallelized exactly where envelope target sets are disjoint — non-overlapping targets cannot conflict, so disjoint groups can apply concurrently while the ledger’s order is assigned deterministically. This is a contention statement *inside* the window, and it is the one place the eliminated tax reappears if disjointness fails. A realization that parallelizes its window has reintroduced a contention graph at frame scale and must reason about it; a realization that keeps the window serial has a ceiling and must measure it.

One period of staging latency. A decision made during the read phase lands in the next window. At headless period — next frame beginning the moment this one completes — the latency is the computation time itself and effectively vanishes. At a fixed sixteen-millisecond frame, decision-to-effect latency is a real constant of up to one frame. Domains that cannot tolerate one period between deciding and affecting are outside the model’s fit, and honesty requires saying so rather than defining the term away. The observation that softens this in practice: interactive simulation already lives at frame granularity everywhere else — input, physics, rendering — so staged behavior adds no granularity the system did not already have.

Scratch is the scope condition on the freeze. Section 3 claimed immutability of *shared* state, and real workers need mutable scratch during the read phase. The model’s requirement is that scratch be private — per worker or per core — so that the claim’s scope is exact. The present realization carries one shared scratch layer in its tile system, inherited from its single-threaded history, in which any consumer writes temporary values and reads results without cleanup. Under the parallel model that layer is a scoped exception, stated here as such: it is either partitioned per consumer before workers share a frame, or it is the one named region excluded from the read phase’s immutability claim. The paper prefers realizations where the code matches the claim exactly, and reports where this one currently stands.

Subversion is now a data race. The prior paper’s nonsubversion rule carried a correctness penalty: a path around the executor is a change the ledger never saw. The frozen frame sharpens the penalty into a harder one. During the read phase, every worker relies on the absence of writers; one bypass — one function that pokes a value directly, at any point in the frame — converts the read phase from immutable to undefined, silently, for every worker at once. Under the frozen frame, a Rule 12 violation does not merely corrupt the audit trail; it corrupts memory. The rule that was an integrity discipline becomes a memory-safety invariant, and the incentive to hold it moves from “the history will be wrong” to “the program will be wrong, intermittently, in ways no test reliably catches” — which any practitioner will recognize as the stronger deterrent.

9. Falsifiable claims

(The test. In this series’ tradition, claims are written to be failed; a realization that cannot fail them is not making them.)

Claim 1 — The synchronization inventory. In a compliant realization, no read during the read phase requires synchronization, and the process’s complete synchronization

inventory is two barriers per frame plus one shutdown flag. Test: audit the realization for locks, atomics, and fences; every instance found must be one of the named three or a defect.

Claim 2 — Constant coordination. Coordination cost per frame is constant in worker count. Test: scale workers from one to the machine’s core count on a fixed workload; measure barrier and dispatch overhead; the curve must be flat within measurement noise, where a conventional job system’s curve is not.

Claim 3 — Replay determinism, conditioned. Given prior frozen state, the delivered envelope set, and the random stream’s state, ledger order and frame output are fully determined. Test: snapshot, run, restore, rerun, diff both the ledger and the state; any divergence is either an uncaptured input (falsifying the realization’s capture) or an unscheduled writer (falsifying Claim 1).

Claim 4 — The properties are consequences, not conventions. Remove either precondition and the structure collapses. Reintroduce a second writer — one function mutating state outside the executor — and data races return, detectable by race-detection tooling that previously reported none. Unstage one mutation — apply a change at its point of decision — and the freeze point dissolves for every consumer of that data. Test: perform each violation deliberately in a branch; the predicted failure class must appear. A realization in which the violations are harmless was never deriving its safety from the rules, and its parallelism is a habit, not a property.

10. Conclusion

(Theory.)

The machinery of concurrent programming — locks, atomics, memory models, dependency graphs — exists to manage the interleaving of reads and writes. The normalization of behavior removes the interleaving instead of managing it. When all change is one verb, staged into windows and applied by a single executor with no path around it, mutation collapses into a scheduled pass, and everything outside that pass reads a world that cannot move. The frozen frame is not a parallelism feature built on the architecture. It is the architecture’s shadow in physical time.

The individual fragments of this property are familiar, because the industry has been recovering them piecemeal for decades: double-buffered state gives phases by convention; entity-component schedulers give declared access enforced by discipline; database snapshot isolation gives frozen reads per transaction; frame graphs give scheduled passes for rendering; persistent data structures give immutability without an economy above it. Each recovers a slice, by mechanism or by habit. The normalization yields the whole property structurally, and the missing element in every neighbor is the same one: nonsubversion — the guarantee not that no one *does* write, but that no unscheduled writer *can exist*. The prior paper called this the difference between a property and a habit, and this paper has shown what that difference is worth when measured in the concurrency tax: four components deleted, the fifth converted into the tunable layout variable that data-oriented practice already knows how to tune. A habit can be forgotten under deadline. A property

cannot — and in the frozen frame, the property is also the memory model, so forgetting it is not a regression but a race, and the rules that were once about audit integrity now hold up the safety of every read on every core.

Bits for what moves; ops for what happens; rows for what changes; and one window for when.

Appendices

Appendix A — The frame timeline

The frame's canonical sequence, referenced from Section 3:

```
┌─ MUTATION WINDOW (serialized) ───────────┐
| 1. Pre-pass: creation envelopes processed |
|      (created entities exist before any entity logic)      |
| 2. Validate + apply each envelope's set, whole-set atomic |
| 3. Ledger appended: applied and refused, ordered, reasoned |
└────────────────── FREEZE POINT ───────────┘
        (last write of the frame)

┌─ READ PHASE (parallel, immutable shared state) ───────────┐
| Workers on cores/nodes read frozen state:                  |
|   scoring · calculations · heat maps · path search ·      |
|   ledger analytics · render prep · UI projection          |
| Outputs are envelopes (staged intents), never writes      |
└────────────────── GATHER POINT ───────────┘
        (emitted envelopes collected)
```

↓

next MUTATION WINDOW

The two barriers of Claim 1 are the freeze point and the gather point. The period between windows is a parameter: sixteen milliseconds rendered, zero headless, nightly in batch domains.

Appendix B — Precondition table

Which rule or principle of [\[HOWL-INFO-17-2026\]](#) supplies which property of the frozen frame. This is the claim-dependency map and the checklist for a reader building a realization in another language: a realization lacking a row's source lacks that row's property.

Source (INFO-17)	Property supplied to the frozen frame
Rule 7 — staged, set-level change	The phases exist: decision and application are separate moments, so a window exists to close
Rule 12 — nonsubversion	The single writer: no unscheduled mutator can exist, so the read phase is immutable rather than usually-unmodified
Principle 1 — quantity at address	Relocatable bytes: state has no code identity, so layout is a free variable
Principle 9 — meaning as assignment	Same, from the meaning side: ids resolve identically anywhere; nothing welded to location
Rule 11 — distribution independence	Placement independence: copy-out/copy-back and NUMA transfer are the small case of scene closure
Principle 10 — inert absence	Partial copies remain valid: sentinels travel as bytes; a partial working set is not an error
Principle 5 — the ledger	The free change feed: notification is a lock-free read of frozen history, replacing the observer pattern
Rule 5 — one interpreter	The window is one stream: no parallel validators or executors to order against each other

Appendix C — Concurrency-tax accounting table

The full version of Section 5's accounting, using the five components of [\[HOWL-INFO-14-2026\]](#) §12.

Component	Conventional engine	Frozen frame: read phase	Frozen frame: mutation window
Contention	Continuous; scales with resource utilization, nonlinear near saturation	Zero by construction — no writer, nothing held	Absent by definition — one stream
Blocking	Threads idle on locks, I/O, reviews; dead budget	Zero — no critical resource exists	Absent — the stream is the schedule
Coordination	Scales with shared-resource count $\times$ access frequency $\times$ protocol cost	Two barriers, constant per frame	None internal
Interleave	Continuous triage; scales with ready-stream count	One dispatch decision per frame	Ledger order is the schedule, decided by arrival
Cascade	Emergent: evictions and invalidations at moments chosen by others	Survives as locality cost — explicit, chosen at dispatch, over immutable data; the tunable variable	—
Topology (per INFO-14 §12)	Varies; stars diverge, hierarchies grow logarithmically	No contention graph exists	Star with one scheduled visit per period — the degenerate, non-growing case

Appendix D — Neighbor comparison

Each neighbor introduced in one sentence, with the slice of the property it recovers and the element it lacks.

Neighbor	What it is	Slice recovered	What it lacks
Double-buffered game state	Two copies of world state; systems read last frame's copy, write this frame's	The phase structure, by convention	No single-writer guarantee: any code may still write anywhere in the write buffer; discipline, not property

Neighbor	What it is	Slice recovered	What it lacks
ECS read/write scheduling	Entity-component systems where each System declares component access and a scheduler orders conflicts	Declared access, conflict-aware ordering	Enforcement is by declaration honesty; N systems remain N private validators and mutators (the prior paper’s Rule 0/5 failure)
MVCC / snapshot isolation	Databases giving each transaction a frozen snapshot while writers proceed	Frozen reads	Per-transaction, not per-world; the application above the database mutates freely; no behavior economy
Frame graphs / render graphs	GPU work declared as a dependency graph of passes over resources	Scheduled passes with declared access	Render-scope only; the game’s own state remains conventionally mutated
Persistent (functional) data structures	Structures whose updates produce new versions, sharing structure	Immutability itself	No economy above it: no guards, staging, ledger, or scoring; immutability without the one verb

The shared gap, stated once: none of the five has **nonsubversion**. Each guarantees, at best, that participants who follow the scheme are safe. None guarantees that no unscheduled writer can exist — and that guarantee is the difference between managing interleaving well and not having interleaving.

Appendix E — Realization notes

(This realization: Zig, Windows, single process, N concurrent games as partition keys. Illustrative throughout; nothing here is theoretically significant, per the prior paper’s register for engineering.)

Window internals. Creation envelopes are processed in a pre-pass so created entities exist before entity logic runs — ordering inside the window is itself staged. Envelopes carry instant or durational delivery in both real seconds and turns, converted by one global constant; the same authored behavior runs turn-based or real-time unmodified, which is the period-as-parameter point of Section 3 in miniature.

Ledger mechanics. The transaction list is append-only with a walk marker (a last-record

flag plus a cached index); records are reused only after a deep reset via a deletion flag, never removed. Entries carry phase, monotonic order, scale, forced flag, applied-versus-inspected flag, first failing reason, and the indices of failed costs and requirements — enough to reconstruct intent; observed per-group deltas are a noted deferral.

Change detection as data. Values update their timestamp only when the value actually changes — writing the same value is not an update — so “time since changed” is queryable on every value, feeding triggers and recency scoring. Tile chunks carry updated-versus-cached timestamps for render-cache invalidation. Both are the frozen frame’s dirty-bit idiom: change information is itself frozen data.

Per-frame memoization. Derived values cache their calculation result against the current frame number in the record itself; within one read phase, repeated reads are one computation. The cache field’s validity is exactly the freeze — the memo cannot go stale inside a phase because its inputs cannot change.

Batch layout. Array of structs per work batch as the default; structure-of-arrays and hot/cold splits applied per workload under profiling, never speculatively. The tile store is chunked row-major arrays (40×40 default), origin-snapped with floor division for negative coordinates.

Thread lifecycle. Long-lived workers created once, pinned by NUMA node (processor-group-aware masks, valid above 64 logical processors) and optional core mask; work arrives as data, not as thread churn. Shutdown is one polled flag, a cooperative drain at batch boundaries with a timeout, and forced termination as last-resort defense against a hung substrate — sufficient cooperatively for the Section 6.3 reason: between batches there is no mid-mutation state to abandon.

Known deltas between realization and claims. One shared scratch tile layer (Section 8’s scoped exception, pending per-consumer partition); randomness-as-state in progress (conditioning Claim 3); the shutdown flag’s atomicity to be made explicit so the audit of Claim 1 finds exactly the named inventory. Listed here because the test section is only honest if the realization’s current distance from it is also on the record.

HOWL-COMP-16-2026. Parallelism as a Consequence of Normalized Behavior: The Frozen Frame — One Verb, One Window, One Writer.

[[HOWL-COMP-16-2026](https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-16-2026)] Parallelism as a Consequence of Normalized Behavior. Github: <https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-16-2026>

[[HOWL-COMP-1-2026](https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-1-2026)] Implementing a Tall-Infra Data-Only Execution System . Github: <https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-1-2026>

[[HOWL-COMP-12-2026](https://github.com/ghowland/papers/tree/main/papers/COMP/COMP-12-2026)] Closed Loop Architecture. Github: <https://github.com/ghowland/papers/tree/main/papers/COMP/COMP-12-2026>

[[HOWL-INFO-11-2026](#)] The Relationship of Zero, One, and Infinity in Information Processing. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-11-2026>

[[HOWL-INFO-14-2026](#)] Bits and Ops. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-14-2026>

[[HOWL-INFO-15-2026](#)] Cross-Domain Learning: How Bits and Ops Transfer Between Domains. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-15-2026>

[[HOWL-INFO-16-2026](#)] Staged Advancement: How Dissolved Knowledge Generates the Next Domain. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-16-2026>

[[HOWL-INFO-17-2026](#)] The General Theory of State Change. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-17-2026>